

[VN] Authentication và Authorization khác nhau thế nào?

[EN] What is the difference between Authentication and Authorization?

[VN] Authentication và Authorization là hai khái niệm quan trọng trong bảo mật hệ thống. Authentication là quá trình xác minh danh tính của người dùng, ví dụ khi người dùng đăng nhập bằng email và mật khẩu để chứng minh họ là ai. Sau khi xác thực thành công, Authorization sẽ xác định người dùng đó được phép làm gì hoặc truy cập tài nguyên nào trong hệ thống, ví dụ chỉ admin mới được chỉnh sửa dữ liệu. Nói đơn giản, Authentication trả lời câu hỏi “Bạn là ai?”, còn Authorization trả lời “Bạn được phép làm gì?”.

[EN] Authentication and Authorization are two important concepts in system security. Authentication is the process of verifying a user's identity, for example when a user logs in with an email and password to prove who they are. After successful authentication, Authorization determines what the user is allowed to do or which resources they can access in the system, such as allowing only admins to edit data. In simple terms, Authentication answers “Who are you?”, while Authorization answers “What are you allowed to do?”.

[VN] JWT là gì và nó hoạt động ra sao?

[EN] What is JWT and how does it work?

[VN] JSON Web Token (JWT) là một chuẩn dùng để xác thực và truyền thông tin an toàn giữa client và server. Khi người dùng đăng nhập thành công, server sẽ tạo một token chứa thông tin người dùng và ký bằng chữ ký số. Token này được gửi về cho client và client sẽ gửi lại token đó trong mỗi request tiếp theo (thường qua header). Server kiểm tra chữ ký của token để xác nhận người dùng hợp lệ mà không cần lưu session trên server, giúp hệ thống hoạt động hiệu quả hơn, đặc biệt trong các ứng dụng web hoặc API.

[EN] JSON Web Token (JWT) is a standard used to authenticate and securely transfer information between a client and a server. When a user logs in successfully, the server creates a token containing user information and signs it with a digital signature. This token is sent back to the client, and the client includes it in future requests (usually in the request header). The server verifies the token's signature to confirm the user's identity without storing a session on the server, which makes the system more efficient, especially for web applications and APIs.

[VN] JWT gồm những phần nào (header, payload, signature)?

[EN] What are the parts of JWT (header, payload, signature)?

[VN] JSON Web Token (JWT) gồm ba phần chính: header, payload và signature. Header chứa thông tin về loại token và thuật toán mã hóa được dùng. Payload chứa dữ liệu của người dùng hoặc các thông tin cần truyền, ví dụ như user ID hoặc quyền truy cập.

Signature là chữ ký số được tạo bằng cách mã hóa header và payload với một secret key, giúp server kiểm tra xem token có bị thay đổi hay không và đảm bảo tính toàn vẹn của dữ liệu.

[EN] A JSON Web Token (JWT) has three main parts: header, payload, and signature. The header contains information about the token type and the encryption algorithm used. The payload stores user data or other information that needs to be shared, such as user ID or permissions. The signature is a digital signature created by encoding the header and payload with a secret key, allowing the server to verify that the token has not been changed and that the data is valid.

[VN] JWT có những rủi ro bảo mật nào?

[EN] What are the security risks of JWT?

[VN] JSON Web Token (JWT) có một số rủi ro bảo mật nếu không được sử dụng đúng cách. Vì token thường được lưu ở phía client, nếu bị đánh cắp (ví dụ qua XSS) thì kẻ tấn công có thể dùng token đó để truy cập hệ thống. Ngoài ra, nếu secret key yếu hoặc cấu hình thuật toán sai, token có thể bị giả mạo. Một rủi ro khác là JWT khó thu hồi (revoke) trước khi hết hạn, vì server thường không lưu session. Do đó, cần dùng HTTPS, thời gian hết hạn ngắn, và bảo vệ nơi lưu token để giảm nguy cơ bảo mật.

[EN] JSON Web Token (JWT) has several security risks if it is not used correctly. Since the token is usually stored on the client side, if it is stolen (for example through XSS) an attacker could use it to access the system. Also, if the secret key is weak or the algorithm is misconfigured, the token might be forged. Another issue is that JWT is hard to revoke before it expires, because the server usually does not store sessions. To reduce these risks, developers should use HTTPS, short expiration times, and secure storage for tokens.

[VN] Khi nào nên dùng JWT vs session-based authentication?

[EN] When should JWT be used vs session-based authentication?

[VN] JSON Web Token (JWT) nên được dùng khi ứng dụng cần hệ thống xác thực không lưu session trên server (stateless), ví dụ trong API, microservices hoặc ứng dụng mobile, nơi client gửi token trong mỗi request. JWT giúp hệ thống dễ mở rộng và hoạt động tốt với nhiều server. Ngược lại, session-based authentication phù hợp với ứng dụng web truyền thống, nơi server lưu session và quản lý trạng thái đăng nhập của người dùng. Cách này thường dễ kiểm soát và dễ thu hồi quyền truy cập hơn khi cần bảo mật chặt chẽ.

[EN] JSON Web Token (JWT) is useful when an application needs stateless authentication, meaning the server does not store user sessions. It is commonly used in APIs, microservices, or mobile applications, where the client sends a token with each request.

JWT helps systems scale easily across multiple servers. In contrast, session-based authentication is better for traditional web applications, where the server stores session data and manages the user's login state. This method is often easier to control and revoke access when strong security is required.

[VN] Access token và Refresh token khác nhau thế nào?

[EN] What is the difference between Access token and Refresh token?

[VN] Trong hệ thống xác thực dùng JSON Web Token (JWT), Access token và Refresh token có vai trò khác nhau. Access token được dùng để truy cập API hoặc tài nguyên bảo vệ và thường có thời gian hết hạn ngắn để giảm rủi ro bảo mật. Khi access token hết hạn, Refresh token sẽ được dùng để yêu cầu server tạo access token mới mà người dùng không cần đăng nhập lại. Refresh token thường có thời gian sống dài hơn và được lưu trữ an toàn hơn để bảo vệ hệ thống.

[EN] In an authentication system using JSON Web Token (JWT), Access token and Refresh token have different roles. An Access token is used to access protected APIs or resources and usually has a short expiration time to reduce security risks. When the access token expires, a Refresh token is used to request a new access token from the server without requiring the user to log in again. Refresh tokens usually have a longer lifetime and are stored more securely to protect the system.

[VN] Tại sao access token nên có expiration time ngắn?

[EN] Why should an access token have a short expiration time?

[VN] Trong hệ thống xác thực sử dụng JSON Web Token (JWT), access token nên có thời gian hết hạn ngắn để giảm rủi ro bảo mật. Nếu token bị đánh cắp, kẻ tấn công chỉ có thể sử dụng nó trong một khoảng thời gian ngắn trước khi token hết hạn. Sau khi hết hạn, người dùng sẽ dùng refresh token để lấy access token mới. Cách này giúp giảm nguy cơ truy cập trái phép và tăng mức độ an toàn cho hệ thống.

[EN] In an authentication system using JSON Web Token (JWT), access tokens should have a short expiration time to reduce security risks. If a token is stolen, the attacker can only use it for a short period before it expires. After it expires, the user can use a refresh token to get a new access token. This approach limits unauthorized access and improves overall system security.

[VN] Làm sao revoke JWT khi user logout?

[EN] How to revoke JWT when user logout?

[VN] Vì JSON Web Token (JWT) thường là stateless, server không lưu session nên việc revoke token khi user logout không đơn giản. Một cách phổ biến là xóa refresh token ở

phía server hoặc database, để người dùng không thể tạo access token mới. Ngoài ra, hệ thống có thể dùng token blacklist để lưu các token đã bị thu hồi cho đến khi chúng hết hạn. Kết hợp với thời gian hết hạn ngắn cho access token, cách này giúp đảm bảo người dùng không thể tiếp tục truy cập sau khi logout.

[EN] Because JSON Web Token (JWT) is usually stateless, the server does not store sessions, so revoking a token when a user logs out is not straightforward. A common solution is to delete the refresh token from the server or database, so the user cannot generate a new access token. Another approach is to use a token blacklist that stores revoked tokens until they expire. Combined with short expiration times for access tokens, this helps ensure the user cannot continue accessing the system after logging out.

[VN] OAuth 2.0 là gì? Nó giải quyết vấn đề gì?

[EN] What is OAuth 2.0 and what problem does it solve?

[VN] OAuth 2.0 là một framework ủy quyền cho phép ứng dụng truy cập tài nguyên của người dùng trên một dịch vụ khác mà không cần chia sẻ mật khẩu. Ví dụ, người dùng có thể đăng nhập vào một ứng dụng bằng tài khoản Google hoặc Facebook. OAuth 2.0 giải quyết vấn đề chia sẻ quyền truy cập an toàn giữa các ứng dụng, bằng cách cấp access token cho ứng dụng thay vì cung cấp thông tin đăng nhập trực tiếp, giúp tăng tính bảo mật và kiểm soát quyền truy cập.

[EN] OAuth 2.0 is an authorization framework that allows an application to access a user's resources from another service without sharing the user's password. For example, users can log in to an app using their Google or Facebook account. OAuth 2.0 solves the problem of securely sharing access between applications by providing an access token to the app instead of giving the user's login credentials, which improves security and access control.

[VN] Các flow phổ biến của OAuth2 là gì?

[EN] What are the common flows of OAuth2?

[VN] Trong OAuth 2.0, có một số flow phổ biến để cấp quyền truy cập. Authorization Code Flow là flow phổ biến nhất, thường dùng cho web app vì an toàn hơn khi server trao đổi code để lấy token. Implicit Flow từng được dùng cho ứng dụng chạy trên trình duyệt, nhưng hiện nay ít dùng vì rủi ro bảo mật. Client Credentials Flow dùng khi server giao tiếp với server mà không cần người dùng đăng nhập. Ngoài ra còn có Resource Owner Password Credentials Flow, nơi ứng dụng nhận trực tiếp username và password, nhưng cách này ít được khuyến khích vì vấn đề bảo mật.

[EN] In OAuth 2.0, there are several common flows used to grant access. Authorization Code Flow is the most common and is usually used for web applications because it is more secure when the server exchanges a code for a token. Implicit Flow was previously used for browser-based applications but is now less recommended due to security risks. Client Credentials Flow is used for server-to-server communication where no user login is required. Another flow is Resource Owner Password Credentials Flow, where the app receives the user's username and password directly, but this method is rarely recommended because of security concerns.

[VN] OAuth2 vs OpenID Connect khác nhau thế nào?

[EN] What is the difference between OAuth2 and OpenID Connect?

[VN] OAuth 2.0 và OpenID Connect có mục đích khác nhau. OAuth 2.0 chủ yếu dùng để ủy quyền (authorization), tức là cho phép một ứng dụng truy cập tài nguyên của người dùng trên một dịch vụ khác thông qua access token. Trong khi đó, OpenID Connect (OIDC) là một lớp xác thực (authentication) được xây dựng trên OAuth 2.0, giúp ứng dụng xác minh danh tính của người dùng và nhận thông tin người dùng thông qua ID token. Nói đơn giản, OAuth 2.0 dùng để cấp quyền truy cập, còn OpenID Connect dùng để xác thực người dùng.

[EN] OAuth 2.0 and OpenID Connect serve different purposes. OAuth 2.0 is mainly used for authorization, meaning it allows an application to access a user's resources on another service using an access token. In contrast, OpenID Connect (OIDC) is an authentication layer built on top of OAuth 2.0 that allows applications to verify the user's identity and receive user information through an ID token. In simple terms, OAuth 2.0 is used to grant access, while OpenID Connect is used to authenticate users.

[VN] Khi nào nên dùng API key authentication?

[EN] When should API key authentication be used?

[VN] API key authentication nên được dùng khi cần xác định ứng dụng hoặc dịch vụ gọi API, thay vì xác thực người dùng cá nhân. Một API key là một chuỗi ký tự bí mật được gửi kèm trong request để server nhận biết và kiểm soát quyền truy cập. Cách này thường được dùng cho public APIs, dịch vụ server-to-server, hoặc khi cần giới hạn số lượng request (rate limiting). Tuy nhiên, API key thường đơn giản nhưng ít bảo mật hơn so với các phương pháp như JSON Web Token hoặc OAuth 2.0, nên chỉ phù hợp với các trường hợp không yêu cầu xác thực người dùng phức tạp.

[EN] API key authentication is used when the goal is to identify the application or service calling an API, rather than authenticating individual users. An API key is a secret string sent with each request so the server can recognize the client and control access. This

method is commonly used for public APIs, server-to-server services, or when applying rate limiting. However, API keys are simple but less secure compared to methods like JSON Web Token or OAuth 2.0, so they are best used in cases that do not require complex user authentication.

[VN] Rate limiting là gì và tại sao cần nó?

[EN] What is rate limiting and why is it needed?

[VN] Rate limiting là kỹ thuật giới hạn số lượng request mà một người dùng hoặc ứng dụng có thể gửi đến API trong một khoảng thời gian nhất định. Mục đích của rate limiting là bảo vệ hệ thống khỏi việc gửi quá nhiều request, giúp tránh quá tải server, giảm nguy cơ tấn công như spam hoặc brute-force, và đảm bảo tài nguyên được chia sẻ công bằng giữa các người dùng. Rate limiting thường được áp dụng trong các hệ thống API hoặc dịch vụ web để giữ cho hệ thống ổn định và an toàn.

[EN] Rate limiting is a technique used to limit the number of requests a user or application can send to an API within a specific time period. The main purpose of rate limiting is to protect the system from too many requests, which helps prevent server overload, reduce risks such as spam or brute-force attacks, and ensure resources are shared fairly among users. Rate limiting is commonly applied in API systems and web services to keep the system stable and secure.

[VN] Các thuật toán rate limiting phổ biến là gì?

[EN] What are the common rate limiting algorithms?

[VN] Rate limiting thường được thực hiện bằng một số thuật toán phổ biến. Fixed Window giới hạn số request trong một khoảng thời gian cố định, ví dụ 100 request mỗi phút. Sliding Window cải tiến hơn bằng cách tính request dựa trên thời gian trượt, giúp kiểm soát chính xác hơn. Token Bucket cho phép request được gửi khi còn “token” trong bucket và token sẽ được thêm lại theo thời gian, giúp hệ thống linh hoạt khi có nhiều request đột ngột. Leaky Bucket xử lý request với tốc độ ổn định bằng cách cho request “chảy ra” từ hàng đợi theo tốc độ cố định. Các thuật toán này giúp hệ thống API ổn định và tránh bị quá tải.

[EN] Rate limiting is usually implemented using several common algorithms. Fixed Window limits the number of requests within a fixed time period, such as 100 requests per minute. Sliding Window improves this by counting requests in a moving time window, providing more accurate control. Token Bucket allows requests only when tokens are available in a bucket, and tokens are refilled over time, making the system flexible during sudden traffic spikes. Leaky Bucket processes requests at a constant rate by letting them

“leak” from a queue steadily. These algorithms help keep API systems stable and prevent overload.

[VN] Rate limiting nên đặt ở API Gateway hay service level?

[EN] Should rate limiting be applied at the API Gateway or service level?

[VN] Rate limiting thường nên được đặt ở API Gateway vì gateway là điểm vào chung của toàn bộ hệ thống, nên có thể kiểm soát và chặn các request quá nhiều trước khi chúng tới các service phía sau, giúp giảm tải cho hệ thống. Tuy nhiên, trong một số trường hợp, rate limiting cũng có thể đặt ở service level để kiểm soát chi tiết hơn cho từng service hoặc chức năng cụ thể. Vì vậy, cách phổ biến là kết hợp cả hai, dùng gateway để bảo vệ toàn hệ thống và dùng service level để kiểm soát logic riêng của từng service.

[EN] Rate limiting is usually placed at the API Gateway because the gateway is the main entry point of the system, so it can control and block excessive requests before they reach backend services, reducing system load. However, in some cases, rate limiting can also be applied at the service level to provide more detailed control for specific services or features. Therefore, a common approach is to combine both, using the gateway to protect the whole system and service-level limits for more specific control.

[VN] Throttling vs rate limiting khác nhau thế nào?

[EN] What is the difference between throttling and rate limiting?

[VN] Rate limiting và throttling đều được dùng để kiểm soát số lượng request đến hệ thống, nhưng cách hoạt động khác nhau. Rate limiting đặt ra giới hạn tối đa số request trong một khoảng thời gian, ví dụ 100 request mỗi phút; nếu vượt quá, request sẽ bị từ chối. Trong khi đó, throttling sẽ giảm tốc độ xử lý request khi lưu lượng quá cao, thay vì chặn ngay lập tức. Nói đơn giản, rate limiting giới hạn số lượng request, còn throttling điều chỉnh tốc độ xử lý để hệ thống không bị quá tải.

[EN] Rate limiting and throttling are both used to control the number of requests sent to a system, but they work differently. Rate limiting sets a maximum number of requests allowed within a time period, such as 100 requests per minute, and requests beyond that limit are rejected. Throttling, on the other hand, slows down the processing of requests when traffic is too high instead of immediately blocking them. In simple terms, rate limiting limits how many requests are allowed, while throttling controls the speed of request processing to prevent system overload.

[VN] Blacklist và Whitelist là gì?

[EN] What are Blacklist and Whitelist?

[VN] Blacklist và Whitelist là hai cách để kiểm soát quyền truy cập trong hệ thống. Blacklist là danh sách các đối tượng bị chặn, ví dụ IP, user hoặc ứng dụng nằm trong danh sách này sẽ không được phép truy cập. Ngược lại, Whitelist là danh sách các đối tượng được phép truy cập, chỉ những đối tượng trong danh sách mới được sử dụng hệ thống hoặc tài nguyên. Nói đơn giản, blacklist chặn những thứ bị cấm, còn whitelist chỉ cho phép những thứ đã được cho phép trước.

[EN] Blacklist and Whitelist are two methods used to control access in a system. A Blacklist is a list of blocked entities, such as IP addresses, users, or applications that are not allowed to access the system. In contrast, a Whitelist is a list of allowed entities, where only the items on the list are permitted to access the system or resources. In simple terms, a blacklist blocks specific items, while a whitelist only allows predefined items.

[VN] Khi nào nên dùng IP whitelist?

[EN] When should IP whitelist be used?

[VN] IP whitelist nên được dùng khi hệ thống chỉ muốn cho phép một số địa chỉ IP đáng tin cậy truy cập vào dịch vụ hoặc API. Cách này thường áp dụng cho hệ thống nội bộ, server-to-server communication, admin panel hoặc các API quan trọng, nơi chỉ có một số server hoặc mạng xác định được phép truy cập. Bằng cách chỉ cho phép các IP đã đăng ký trước, IP whitelist giúp tăng mức độ bảo mật và giảm nguy cơ truy cập trái phép.

[EN] IP whitelist should be used when a system wants to allow access only from specific trusted IP addresses. This is commonly used for internal systems, server-to-server communication, admin panels, or important APIs, where only certain servers or networks should have access. By allowing only pre-approved IP addresses, IP whitelisting improves security and reduces the risk of unauthorized access.

[VN] Làm sao bảo vệ API khỏi brute-force attack?

[EN] How to protect an API from brute-force attacks?

[VN] Để bảo vệ API khỏi brute-force attack (tấn công thử nhiều lần để đoán mật khẩu hoặc token), hệ thống có thể áp dụng nhiều biện pháp bảo mật. Ví dụ, sử dụng rate limiting để giới hạn số request trong một khoảng thời gian, khóa tạm thời tài khoản sau nhiều lần đăng nhập sai, và áp dụng IP blacklist hoặc IP whitelist để chặn các nguồn đáng ngờ. Ngoài ra, nên dùng CAPTCHA, yêu cầu mật khẩu mạnh, và ghi log để phát hiện hành vi bất thường. Những biện pháp này giúp giảm khả năng kẻ tấn công thử quá nhiều lần và bảo vệ hệ thống tốt hơn.

[EN] To protect an API from a brute-force attack (where attackers repeatedly try many passwords or tokens), several security measures can be applied. For example, use rate

limiting to limit the number of requests in a period of time, temporarily lock accounts after multiple failed login attempts, and apply IP blacklist or IP whitelist to block suspicious sources. In addition, systems can use CAPTCHA, require strong passwords, and log activities to detect unusual behavior. These methods help reduce the chance of repeated attack attempts and improve overall API security.

[VN] Làm sao bảo vệ API khỏi credential stuffing?

[EN] How to protect an API from credential stuffing?

[VN] Để bảo vệ API khỏi credential stuffing (tấn công dùng danh sách username và password bị lộ để đăng nhập hàng loạt), hệ thống cần áp dụng nhiều biện pháp bảo mật. Ví dụ, sử dụng rate limiting để giới hạn số lần đăng nhập, phát hiện và chặn các IP đáng ngờ, và khóa tạm thời tài khoản sau nhiều lần đăng nhập thất bại. Ngoài ra, nên áp dụng multi-factor authentication (MFA), yêu cầu mật khẩu mạnh, và theo dõi hoạt động đăng nhập bất thường. Những biện pháp này giúp ngăn kẻ tấn công thử nhiều thông tin đăng nhập bị rò rỉ trên hệ thống.

[EN] To protect an API from credential stuffing (an attack where leaked username and password lists are used to attempt many logins), several security measures should be applied. For example, use rate limiting to limit login attempts, detect and block suspicious IP addresses, and temporarily lock accounts after multiple failed login attempts. In addition, systems should use multi-factor authentication (MFA), require strong passwords, and monitor unusual login activity. These measures help prevent attackers from testing large numbers of stolen credentials on the system.

[VN] SQL Injection là gì và cách phòng tránh?

[EN] What is SQL Injection and how to prevent it?

[VN] SQL Injection là một lỗ hổng bảo mật xảy ra khi kẻ tấn công chèn câu lệnh SQL độc hại vào input của người dùng để truy cập, thay đổi hoặc xóa dữ liệu trong database. Điều này thường xảy ra khi ứng dụng ghép chuỗi trực tiếp từ input vào câu lệnh SQL mà không kiểm tra hoặc lọc dữ liệu. Để phòng tránh, lập trình viên nên dùng prepared statements hoặc parameterized queries, kiểm tra và lọc input của người dùng, và hạn chế quyền truy cập của database. Những cách này giúp ngăn chặn việc thực thi các câu lệnh SQL độc hại.

[EN] SQL Injection is a security vulnerability where an attacker inserts malicious SQL code into user input to access, modify, or delete data in a database. This usually happens when an application directly combines user input into SQL queries without proper validation or filtering. To prevent this, developers should use prepared statements or parameterized queries, validate and sanitize user input, and limit database permissions. These methods help stop malicious SQL commands from being executed.

[VN] XSS (Cross-site scripting) là gì?

[EN] What is XSS (Cross-site scripting)?

[VN] Cross-site scripting (XSS) là một lỗ hổng bảo mật xảy ra khi kẻ tấn công chèn mã JavaScript độc hại vào trang web mà người dùng khác truy cập. Khi trang web hiển thị nội dung này mà không kiểm tra hoặc lọc dữ liệu, đoạn script có thể chạy trong trình duyệt của người dùng và đánh cắp cookie, token hoặc thông tin cá nhân. Để phòng tránh XSS, lập trình viên nên lọc và encode dữ liệu đầu vào, không render trực tiếp HTML từ user input, và sử dụng các cơ chế bảo mật của trình duyệt như Content Security Policy (CSP).

[EN] Cross-site scripting (XSS) is a security vulnerability that occurs when an attacker injects malicious JavaScript code into a website that other users visit. If the website displays this content without proper validation or filtering, the script can run in the user's browser and steal cookies, tokens, or personal information. To prevent XSS, developers should sanitize and encode user input, avoid rendering raw HTML from user data, and use browser security features such as Content Security Policy (CSP).

[VN] CSRF attack là gì và cách phòng tránh?

[EN] What is a CSRF attack and how to prevent it?

[VN] Cross-Site Request Forgery (CSRF) là một kiểu tấn công trong đó kẻ tấn công lừa người dùng đã đăng nhập gửi request không mong muốn đến server. Vì trình duyệt tự động gửi cookie hoặc session, server có thể nghĩ rằng request đó là hợp lệ từ người dùng. Điều này có thể dẫn đến các hành động nguy hiểm như thay đổi mật khẩu hoặc chuyển tiền. Để phòng tránh CSRF, lập trình viên thường sử dụng CSRF token, kiểm tra Origin hoặc Referer header, và dùng SameSite cookie để giảm nguy cơ request giả mạo.

[EN] Cross-Site Request Forgery (CSRF) is an attack where an attacker tricks a logged-in user into sending an unwanted request to a server. Since browsers automatically include cookies or session data, the server may think the request is legitimate. This can lead to dangerous actions such as changing a password or making transactions. To prevent CSRF, developers commonly use CSRF tokens, check the Origin or Referer headers, and apply SameSite cookies to reduce the risk of forged requests.

[VN] Tại sao cần HTTPS / TLS cho API?

[EN] Why is HTTPS / TLS needed for APIs?

[VN] HTTPS sử dụng Transport Layer Security để mã hóa dữ liệu truyền giữa client và server. Điều này rất quan trọng cho API vì nó giúp bảo vệ thông tin nhạy cảm như mật khẩu, token hoặc dữ liệu cá nhân khỏi bị nghe lén hoặc thay đổi bởi kẻ tấn công trong quá trình truyền dữ liệu. Ngoài ra, HTTPS/TLS còn giúp xác minh danh tính của server và đảm

bảo dữ liệu không bị chỉnh sửa. Vì vậy, sử dụng HTTPS/TLS là cần thiết để tăng độ an toàn và độ tin cậy cho hệ thống API.

[EN] HTTPS uses Transport Layer Security to encrypt data sent between the client and the server. This is very important for APIs because it helps protect sensitive information such as passwords, tokens, or personal data from being intercepted or modified by attackers during transmission. HTTPS/TLS also verifies the server's identity and ensures the data has not been altered. Therefore, using HTTPS/TLS is essential to improve the security and reliability of API systems.

[VN] CORS là gì và tại sao cần nó?

[EN] What is CORS and why is it needed?

[VN] Cross-Origin Resource Sharing (CORS) là một cơ chế bảo mật của trình duyệt cho phép kiểm soát việc một trang web có thể truy cập tài nguyên từ domain khác hay không. Theo mặc định, trình duyệt áp dụng same-origin policy, nghĩa là một website chỉ được gọi API từ cùng domain. CORS cho phép server chỉ định domain nào được phép truy cập thông qua các HTTP header. Điều này giúp bảo vệ API khỏi các request trái phép từ website khác, đồng thời vẫn cho phép các ứng dụng frontend và backend ở domain khác nhau hoạt động cùng nhau.

[EN] Cross-Origin Resource Sharing (CORS) is a browser security mechanism that controls whether a website can access resources from a different domain. By default, browsers follow the same-origin policy, which means a website can only call APIs from the same domain. CORS allows the server to specify which domains are allowed to access its resources using HTTP headers. This helps protect APIs from unauthorized requests from other websites, while still allowing frontend and backend applications on different domains to work together.

[VN] SameSite cookie giúp chống CSRF như thế nào?

[EN] How does SameSite cookie help prevent CSRF?

[VN] SameSite cookie giúp chống Cross-Site Request Forgery bằng cách kiểm soát khi nào cookie được gửi cùng với request từ website khác (cross-site request). Khi cookie được đặt với thuộc tính SameSite=Strict hoặc SameSite=Lax, trình duyệt sẽ không gửi cookie trong các request đến từ domain khác, vì vậy server sẽ không nhận được session của người dùng. Điều này ngăn kẻ tấn công lợi dụng trình duyệt của người dùng để gửi request giả mạo đến server.

[EN] The SameSite cookie helps prevent Cross-Site Request Forgery by controlling when cookies are sent with cross-site requests. When a cookie is set with SameSite=Strict or

SameSite=Lax, the browser does not send the cookie with requests coming from another domain. As a result, the server does not receive the user's session information, which prevents attackers from using the user's browser to send forged requests to the server.

[VN] Tại sao JWT không nên chứa dữ liệu nhạy cảm?

[EN] Why should JWT not contain sensitive data?

[VN] Trong JSON Web Token (JWT), dữ liệu trong payload không được mã hóa mặc định mà chỉ được encode, nên bất kỳ ai có token cũng có thể giải mã và đọc nội dung bên trong. Vì vậy, nếu JWT chứa dữ liệu nhạy cảm như mật khẩu, thông tin cá nhân hoặc secret key, thông tin đó có thể bị lộ khi token bị đánh cắp hoặc bị truy cập trái phép. Do đó, JWT chỉ nên chứa thông tin cần thiết và không nhạy cảm, ví dụ như user ID hoặc role.

[EN] In JSON Web Token (JWT), the data in the payload is not encrypted by default, it is only encoded, which means anyone who has the token can decode and read its contents. Because of this, if a JWT contains sensitive information such as passwords, personal data, or secret keys, that data could be exposed if the token is stolen or accessed by attackers. Therefore, a JWT should only include necessary and non-sensitive information, such as a user ID or role.

[VN] JWT nên lưu ở đâu? Cookie vs LocalStorage

[EN] Where should JWT be stored? Cookie vs LocalStorage

[VN] Khi sử dụng JSON Web Token (JWT), token có thể được lưu trong cookie hoặc localStorage, nhưng mỗi cách có ưu và nhược điểm. Lưu JWT trong localStorage dễ sử dụng cho ứng dụng frontend, nhưng có nguy cơ bị đánh cắp nếu xảy ra Cross-site scripting (XSS). Ngược lại, lưu JWT trong HTTP-only cookie an toàn hơn trước XSS vì JavaScript không thể truy cập cookie này, nhưng cần cấu hình thêm để tránh Cross-Site Request Forgery (CSRF). Vì vậy, trong nhiều hệ thống, cách phổ biến là lưu access token tạm thời ở client và lưu refresh token trong HTTP-only cookie để tăng bảo mật.

[EN] When using JSON Web Token (JWT), the token can be stored in cookies or localStorage, and each option has advantages and disadvantages. Storing JWT in localStorage is simple for frontend applications, but it can be stolen if a Cross-site scripting (XSS) attack occurs. On the other hand, storing JWT in an HTTP-only cookie is safer against XSS because JavaScript cannot access the cookie, but it requires extra protection against Cross-Site Request Forgery (CSRF). Therefore, many systems use a hybrid approach: store the access token temporarily on the client and keep the refresh token in an HTTP-only cookie for better security.

[VN] Tại sao refresh token nên lưu an toàn hơn access token?

[EN] Why should refresh tokens be stored more securely than access tokens?

[VN] Trong hệ thống dùng JSON Web Token (JWT), refresh token cần được lưu an toàn hơn access token vì refresh token thường có thời gian sống dài hơn và có thể được dùng để tạo ra nhiều access token mới. Nếu kẻ tấn công đánh cắp refresh token, họ có thể tiếp tục lấy access token mới và truy cập hệ thống trong thời gian dài. Trong khi đó, access token thường có thời gian hết hạn ngắn, nên nếu bị lộ thì mức độ rủi ro thấp hơn. Vì vậy, refresh token thường được lưu trong HTTP-only cookie hoặc hệ thống lưu trữ an toàn để giảm nguy cơ bị đánh cắp bởi các lỗ hổng như Cross-site scripting (XSS).

[EN] In systems using JSON Web Token (JWT), the refresh token must be stored more securely than the access token because refresh tokens usually have a longer lifetime and can be used to generate many new access tokens. If an attacker steals a refresh token, they can keep creating new access tokens and access the system for a long time. In contrast, access tokens usually have a short expiration time, so the risk is lower if they are exposed. Therefore, refresh tokens are often stored in HTTP-only cookies or other secure storage to reduce the risk of being stolen through vulnerabilities such as Cross-site scripting (XSS).

[VN] Token rotation là gì?

[EN] What is token rotation?

[VN] Token rotation là kỹ thuật bảo mật trong đó mỗi lần dùng JSON Web Token (JWT) để làm mới phiên đăng nhập, hệ thống sẽ tạo một refresh token mới và vô hiệu hóa refresh token cũ. Điều này giúp giảm rủi ro nếu refresh token bị đánh cắp, vì token cũ sẽ không còn sử dụng được sau khi đã được thay thế. Nhờ vậy, token rotation giúp tăng bảo mật cho hệ thống xác thực và hạn chế việc kẻ tấn công dùng lại token đã bị lộ.

[EN] Token rotation is a security technique where each time a JSON Web Token (JWT) is used to refresh a session, the system generates a new refresh token and invalidates the old one. This reduces risk if a refresh token is stolen, because the old token will no longer work after it is replaced. As a result, token rotation improves authentication security and prevents attackers from reusing leaked tokens.

[VN] Làm sao phát hiện token replay attack?

[EN] How to detect a token replay attack?

[VN] Token replay attack xảy ra khi kẻ tấn công lấy được token hợp lệ và gửi lại nhiều lần để truy cập hệ thống. Để phát hiện điều này trong hệ thống dùng JSON Web Token (JWT), có thể theo dõi ID của token (jti), ghi log các lần sử dụng token, và phát hiện khi một token được dùng nhiều lần bất thường hoặc từ các địa chỉ IP / thiết bị khác nhau trong thời gian ngắn. Ngoài ra, có thể dùng token rotation, thời gian hết hạn ngắn cho access token, và lưu trạng thái token trên server để phát hiện nếu một token cũ bị dùng lại.

[EN] A token replay attack happens when an attacker steals a valid token and sends it again many times to access the system. To detect this in systems using JSON Web Token (JWT), the server can track the token ID (jti), log token usage, and detect unusual behavior such as the same token being used many times or from different IP addresses or devices in a short time. Other methods include using token rotation, short expiration time for access tokens, and storing token state on the server to detect when an old token is reused.

[VN] Làm sao authenticate giữa các microservices?

[EN] How to authenticate between microservices?

[VN] Để xác thực giữa các microservices, các dịch vụ thường sử dụng token bảo mật hoặc service identity. Một cách phổ biến là dùng JSON Web Token (JWT): service gọi API sẽ gửi JWT trong request, và service nhận sẽ kiểm tra chữ ký của token để xác thực. Ngoài ra, hệ thống có thể dùng Mutual TLS (mTLS), trong đó cả client và server đều dùng chứng chỉ TLS để xác minh lẫn nhau. Cách này giúp đảm bảo rằng chỉ những service hợp lệ trong hệ thống mới có thể giao tiếp với nhau.

[EN] To authenticate between microservices, services usually use security tokens or service identity. A common method is using JSON Web Token (JWT): the calling service sends a JWT in the request, and the receiving service verifies the token signature to confirm identity. Another method is Mutual TLS (mTLS), where both the client and server use TLS certificates to verify each other. This ensures that only trusted services inside the system can communicate with each other.

[VN] mTLS (mutual TLS) là gì?

[EN] What is mTLS (mutual TLS)?

[VN] Mutual TLS (mTLS) là một cơ chế bảo mật dựa trên Transport Layer Security (TLS), trong đó cả client và server đều phải xác thực lẫn nhau bằng chứng chỉ số. Trong TLS thông thường, chỉ server gửi chứng chỉ để client xác minh. Nhưng trong mTLS, client cũng phải gửi chứng chỉ của mình để server kiểm tra. Nhờ vậy, mTLS giúp đảm bảo rằng cả hai bên đều đáng tin cậy, và thường được dùng để bảo mật giao tiếp giữa các service trong hệ thống microservices.

[EN] Mutual TLS (mTLS) is a security method based on Transport Layer Security (TLS) where both the client and the server authenticate each other using digital certificates. In normal TLS, only the server sends a certificate for the client to verify. In mTLS, the client also sends its certificate so the server can verify it. This ensures that both sides are trusted, and it is commonly used to secure communication between services in a microservices system.

[VN] Service mesh giúp bảo mật microservices như thế nào?

[EN] How does a service mesh help secure microservices?

[VN] Service Mesh là một lớp hạ tầng giúp quản lý và bảo mật việc giao tiếp giữa các microservices. Nó thường tự động áp dụng Mutual TLS (mTLS) để mã hóa dữ liệu và xác thực giữa các service, giúp ngăn chặn truy cập trái phép. Ngoài ra, service mesh còn hỗ trợ kiểm soát truy cập, theo dõi lưu lượng (traffic), ghi log và phát hiện bất thường, giúp hệ thống microservices an toàn và dễ quản lý hơn.

[EN] A Service Mesh is an infrastructure layer that manages and secures communication between microservices. It often automatically applies Mutual TLS (mTLS) to encrypt data and authenticate services, which helps prevent unauthorized access. In addition, a service mesh can provide access control, traffic monitoring, logging, and anomaly detection, making the microservices system more secure and easier to manage.

[VN] Secret management nên làm thế nào trong production?

[EN] How should secret management be handled in production?

[VN] Trong môi trường production, secret management nên được quản lý tập trung bằng các hệ thống chuyên dụng thay vì lưu trực tiếp trong code hoặc file cấu hình. Các bí mật như mật khẩu, API key hoặc token nên được lưu trong công cụ quản lý bí mật như HashiCorp Vault, AWS Secrets Manager, hoặc Azure Key Vault. Những công cụ này giúp mã hóa dữ liệu, kiểm soát quyền truy cập, tự động xoay vòng (rotate) secret và ghi log truy cập, từ đó giảm nguy cơ lộ thông tin nhạy cảm và tăng độ an toàn cho hệ thống.

[EN] In production environments, secret management should be handled by dedicated systems instead of storing secrets directly in code or configuration files. Sensitive data such as passwords, API keys, or tokens should be stored in secret management tools like HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault. These tools help encrypt secrets, control access permissions, automatically rotate secrets, and log access, which reduces the risk of data leaks and improves system security.

[VN] Tại sao không nên lưu secret trong source code?

[EN] Why should secrets not be stored in source code?

[VN] Không nên lưu secret (như mật khẩu, API key, token) trong source code vì khi code được đưa lên hệ thống quản lý phiên bản như Git hoặc GitHub, những thông tin này có thể bị lộ cho người khác. Ngay cả khi sau đó bạn xóa secret khỏi code, nó vẫn có thể tồn tại trong lịch sử commit. Nếu kẻ tấn công lấy được secret, họ có thể truy cập vào hệ thống hoặc dữ liệu quan trọng. Vì vậy, secret nên được lưu trong các hệ thống quản lý bí mật hoặc biến môi trường (environment variables) để tăng tính bảo mật.

[EN] Secrets (such as passwords, API keys, and tokens) should not be stored in source code because when the code is uploaded to version control systems like Git or platforms like GitHub, the secrets may be exposed to others. Even if the secret is removed later, it can still remain in the commit history. If an attacker gets the secret, they may access the system or sensitive data. Therefore, secrets should be stored in secret management systems or environment variables to improve security.

[VN] IAM role vs IAM user trong AWS khác nhau thế nào?

[EN] What is the difference between IAM role and IAM user in AWS?

[VN] Trong AWS Identity and Access Management (IAM) của Amazon Web Services, IAM user là một danh tính cố định dùng cho một người hoặc ứng dụng, có username, password hoặc access key để truy cập tài nguyên AWS. Ngược lại, IAM role không gắn với một người cụ thể mà là một tập quyền (permissions) có thể được tạm thời “assume” bởi user, service hoặc ứng dụng, và thường dùng cho EC2, Lambda hoặc cross-account access. Vì role dùng credentials tạm thời, nên an toàn hơn và thường được khuyến nghị dùng trong production.

[EN] In AWS Identity and Access Management (IAM) from Amazon Web Services, an IAM user is a fixed identity for a person or application, with a username, password, or access keys to access AWS resources. In contrast, an IAM role is not tied to a specific person; it is a set of permissions that can be temporarily assumed by users, services, or applications. Roles are often used by services like EC2, Lambda, or for cross-account access. Because roles use temporary credentials, they are generally more secure and recommended for production environments.

[VN] Principle of least privilege là gì?

[EN] What is the principle of least privilege?

[VN] Principle of Least Privilege là nguyên tắc bảo mật nói rằng người dùng, ứng dụng hoặc hệ thống chỉ nên được cấp quyền tối thiểu cần thiết để thực hiện công việc của mình, không nhiều hơn. Điều này giúp giảm rủi ro nếu tài khoản hoặc hệ thống bị tấn công, vì kẻ tấn công sẽ chỉ có quyền hạn chế. Nguyên tắc này thường được áp dụng trong hệ thống quản lý quyền truy cập như AWS Identity and Access Management để tăng tính an toàn cho hệ thống.

[EN] The Principle of Least Privilege is a security rule that says users, applications, or systems should only be given the minimum permissions needed to perform their tasks, and nothing more. This reduces risk if an account or system is compromised, because an attacker will only have limited access. This principle is commonly applied in access

control systems such as AWS Identity and Access Management to improve overall system security.

[VN] Làm sao quản lý secrets trong Kubernetes?

[EN] How to manage secrets in Kubernetes?

[VN] Trong Kubernetes, secrets được quản lý bằng Kubernetes Secrets, một loại resource dùng để lưu dữ liệu nhạy cảm như mật khẩu, API key hoặc token. Các secrets này có thể được mount vào container dưới dạng environment variables hoặc file để ứng dụng sử dụng mà không cần ghi trực tiếp trong code. Để tăng bảo mật trong production, secrets nên được mã hóa trong etcd, giới hạn quyền truy cập bằng RBAC và có thể tích hợp với hệ thống quản lý secret bên ngoài như HashiCorp Vault.

[EN] In Kubernetes, secrets are managed using Kubernetes Secrets, a resource designed to store sensitive data such as passwords, API keys, or tokens. These secrets can be mounted into containers as environment variables or files, allowing applications to use them without storing them directly in the code. For better security in production, secrets should be encrypted in etcd, access should be controlled with RBAC, and they can be integrated with external secret managers like HashiCorp Vault.

[VN] Tại sao cần network policies trong Kubernetes?

[EN] Why are network policies needed in Kubernetes?

[VN] Trong Kubernetes, Kubernetes Network Policy được dùng để kiểm soát pod nào có thể giao tiếp với pod khác hoặc với dịch vụ bên ngoài. Mặc định, nhiều cluster cho phép tất cả pod giao tiếp với nhau, điều này có thể gây rủi ro bảo mật. Network policies giúp giới hạn lưu lượng mạng theo nguyên tắc Principle of Least Privilege, chỉ cho phép các kết nối cần thiết. Nhờ đó, nếu một pod bị tấn công, kẻ tấn công sẽ khó truy cập sang các dịch vụ khác trong hệ thống.

[EN] In Kubernetes, Kubernetes Network Policy is used to control which pods can communicate with other pods or external services. By default, many clusters allow all pods to communicate with each other, which can create security risks. Network policies help limit network traffic based on the Principle of Least Privilege, allowing only necessary connections. This means if one pod is compromised, it becomes harder for an attacker to access other services in the system.

[VN] AWS Security Group vs NACL khác nhau thế nào?

[EN] What is the difference between AWS Security Group and NACL?

[VN] Trong Amazon Web Services, AWS Security Group và AWS Network ACL đều dùng để kiểm soát traffic mạng nhưng hoạt động khác nhau. Security Group hoạt động ở mức

instance (như EC2), là stateful nên nếu một request được cho phép đi vào thì response sẽ tự động được cho phép đi ra. Ngược lại, NACL hoạt động ở mức subnet và là stateless, nghĩa là phải cấu hình rule riêng cho cả inbound và outbound. Security Group thường dùng để kiểm soát chi tiết cho từng instance, còn NACL dùng để tạo lớp bảo mật bổ sung cho toàn bộ subnet.

[EN] In Amazon Web Services, AWS Security Group and AWS Network ACL are both used to control network traffic but work differently. A Security Group works at the instance level (such as EC2) and is stateful, meaning if an incoming request is allowed, the response is automatically allowed out. In contrast, a NACL works at the subnet level and is stateless, so rules must be defined for both inbound and outbound traffic. Security Groups are usually used for detailed control on each instance, while NACLs provide an extra security layer for the whole subnet.

[VN] Làm sao bảo vệ S3 bucket khỏi public access?

[EN] How to protect an S3 bucket from public access?

[VN] Để bảo vệ Amazon S3 bucket khỏi truy cập công khai, trước hết nên bật S3 Block Public Access để chặn tất cả cấu hình public. Ngoài ra, cần cấu hình AWS Identity and Access Management policy chỉ cho phép những user hoặc role cần thiết truy cập bucket. Cũng nên kiểm tra bucket policy và ACL để đảm bảo không có rule cho phép truy cập public, đồng thời có thể bật logging để theo dõi các truy cập bất thường.

[EN] To protect an Amazon S3 bucket from public access, first enable S3 Block Public Access to block all public configurations. You should also configure AWS Identity and Access Management policies so that only required users or roles can access the bucket. In addition, review the bucket policy and ACL to ensure there are no public rules, and enable logging to monitor unusual access.

[VN] Làm sao quản lý secrets an toàn trong Python app?

[EN] How to securely manage secrets in a Python app?

[VN] Để quản lý secrets an toàn trong Python app, bạn không nên hard-code mật khẩu, API key hay secret key trong source code hoặc commit lên Git. Thay vào đó, nên lưu secrets trong environment variables, file cấu hình được bảo vệ hoặc dùng secret manager như AWS Secrets Manager, HashiCorp Vault. Trong production, chỉ cấp quyền truy cập tối thiểu cho service và bật cơ chế mã hóa khi lưu trữ và truyền dữ liệu. Ngoài ra, cần xoay vòng (rotate) secrets định kỳ và giới hạn quyền truy cập để giảm rủi ro rò rỉ.

[EN] To manage secrets securely in a Python app, you should not hard-code passwords, API keys, or secret keys in the source code or commit them to Git. Instead, store secrets in

environment variables, protected configuration files, or use a secret manager such as AWS Secrets Manager or HashiCorp Vault. In production, apply the principle of least privilege and enable encryption for data at rest and in transit. You should also rotate secrets regularly and limit access permissions to reduce the risk of leaks.

[VN] Security best practice khi build Docker image?

[EN] Security best practices when building Docker images?

[VN] Khi build Docker image cho production, cần tuân thủ một số best practice về bảo mật. Thứ nhất, dùng base image nhẹ và chính thức, tránh image không rõ nguồn gốc. Thứ hai, dùng multi-stage build để loại bỏ tool build và file không cần thiết khỏi image cuối. Thứ ba, không chạy container bằng user root mà tạo user riêng với quyền hạn tối thiểu. Thứ tư, không hard-code secret vào image mà dùng environment variable hoặc secret manager. Ngoài ra, nên thường xuyên cập nhật dependency, quét lỗ hổng bảo mật và chỉ mở những port thật sự cần thiết. Những cách này giúp giảm attack surface và tăng độ an toàn cho hệ thống.

[EN] When building a Docker image for production, several security best practices should be followed. First, use a lightweight and official base image, and avoid untrusted images. Second, apply multi-stage build to remove build tools and unnecessary files from the final image. Third, do not run the container as root; instead, create a dedicated user with minimal privileges. Fourth, never hard-code secrets inside the image; use environment variables or a secret manager. Additionally, regularly update dependencies, scan for vulnerabilities, and expose only necessary ports. These practices reduce the attack surface and improve overall system security.

[VN] ConfigMap vs Secret trong Kubernetes.

[EN] ConfigMap vs Secret in Kubernetes.

[VN] ConfigMap và Secret đều dùng để lưu cấu hình trong Kubernetes, nhưng mục đích khác nhau. ConfigMap dùng để lưu dữ liệu cấu hình không nhạy cảm như URL service, feature flag hoặc cấu hình ứng dụng. Secret dùng để lưu dữ liệu nhạy cảm như mật khẩu, API key hoặc token. Secret được mã hóa base64 và có thể tích hợp với hệ thống quản lý secret bên ngoài để tăng bảo mật. Tóm lại, ConfigMap cho cấu hình thông thường, còn Secret cho thông tin bảo mật cần bảo vệ chặt chẽ.

[EN] ConfigMap and Secret are both used to store configuration in Kubernetes, but they serve different purposes. ConfigMap stores non-sensitive configuration data such as service URLs, feature flags, or application settings. Secret stores sensitive data such as passwords, API keys, or tokens. Secrets are encoded in base64 and can integrate with external secret management systems for better security. In summary, ConfigMap is for

general configuration, while Secret is for sensitive information that needs stronger protection.

[VN] IAM role hoạt động thế nào?

[EN] How does an IAM role work?

[VN] IAM role là cơ chế phân quyền trong AWS cho phép một user, service hoặc resource như EC2, Lambda truy cập tài nguyên AWS mà không cần lưu access key cố định trong code. Role chứa một hoặc nhiều policy, trong đó định nghĩa các quyền như được phép đọc S3 hay ghi DynamoDB. Khi một service được gán IAM role, AWS sẽ cấp temporary credentials tự động thông qua cơ chế AssumeRole. Các credential này có thời hạn ngắn và được làm mới định kỳ, giúp tăng bảo mật và giảm rủi ro lộ key.

[EN] IAM role is a permission mechanism in AWS that allows a user, service, or resource such as EC2 or Lambda to access AWS resources without storing permanent access keys in code. A role contains one or more policies that define permissions, such as reading from S3 or writing to DynamoDB. When a service is assigned an IAM role, AWS provides temporary credentials automatically through the AssumeRole process. These credentials are short-lived and refreshed periodically, improving security and reducing the risk of key exposure.

VIII. Security

Authentication và Authorization khác nhau thế nào?

JWT là gì và nó hoạt động ra sao?

JWT gồm những phần nào(header, payload, signature)?

JWT có những rủi ro bảo mật nào?

Khi nào nên dùng JWT vs session-based authentication?

Access token và Refresh token khác nhau thế nào?

Tại sao access token nên có expiration time ngắn?

Làm sao revoke JWT khi user logout?

OAuth 2.0 là gì? Nó giải quyết vấn đề gì?

Các flow phổ biến của OAuth2 là gì?

OAuth2 vs OpenID Connect khác nhau thế nào?

Khi nào nên dùng API key authentication?

Rate limiting là gì và tại sao cần nó?

Các thuật toán rate limiting phổ biến là gì?

Rate limiting nên đặt ở API Gateway hay service level?

Throttling vs rate limiting khác nhau thế nào?

Blacklist và Whitelist là gì?

Khi nào nên dùng IP whitelist?
Làm sao bảo vệ API khỏi brute-force attack?
Làm sao bảo vệ API khỏi credential stuffing?
SQL Injection là gì và cách phòng tránh?
XSS(Cross-site scripting) là gì?
CSRF attack là gì và cách phòng tránh?
Tại sao cần HTTPS / TLS cho API?
CORS là gì và tại sao cần nó?
SameSite cookie giúp chống CSRF như thế nào?
Tại sao JWT không nên chứa dữ liệu nhạy cảm?
JWT nên lưu ở đâu? Cookie vs LocalStorage
Tại sao refresh token nên lưu an toàn hơn access token?
Token rotation là gì?
Làm sao phát hiện token replay attack?
Làm sao authenticate giữa các microservices?
mTLS(mutual TLS) là gì?
Service mesh giúp bảo mật microservices như thế nào?
Secret management nên làm thế nào trong production?
Tại sao không nên lưu secret trong source code?
IAM role vs IAM user trong AWS khác nhau thế nào?
Principle of least privilege là gì?
Làm sao quản lý secrets trong Kubernetes?
Tại sao cần network policies trong Kubernetes?
AWS Security Group vs NACL khác nhau thế nào?
Làm sao bảo vệ S3 bucket khỏi public access?
Làm sao quản lý secrets an toàn trong Python app?
Security best practice khi build Docker image?
ConfigMap vs Secret trong Kubernetes.
IAM role hoạt động thế nào?